PORTER: A Plan-Committed Router for Multi-Core Distributed Quantum Computers

Fuchao Sheng, Yongjian Xie, Yongming Li, Sanjiang Li *

***Abstract*—A distributed quantum computer runs one circuit across several small cores joined by links that distribute entanglement on demand; under the usual cost model the compiler minimises the Einstein–Podolsky–Rosen (EPR) pairs spent on gates that straddle cores. Some existing routers fold into an inter-core action's score the preparation the current layout demands for it, so its apparent value depends on the layout the compiler happens to occupy. We present PORTER, which instead ranks complete inter-core *plans* — which primitive fires, which gates it resolves, which readiness conditions it needs — on the gate progress each buys, letting preparation cost only break ties, then commits to one and prepares it. It supports the three inter-core primitives studied under this cost model, including cat-entangler packets that amortise one pair over a run of remote gates sharing a control, and adds capacity-aware relief, a deterministic initial layout, and a teleport-lookahead horizon chosen per circuit from a fixed four-value portfolio. On MQT-Bench suites at 25, 64 and 80 logical qubits PORTER uses 59.7%, 44.9% and 68.8% fewer EPR pairs in geometric mean than published TELESABRE at four routes to its three, over the instances both complete, and completes all twenty-four instances to TELESABRE's seventeen. Against DSABRE it is competitive on those suites — $0.99\times$ its pooled EPR at $1.20\times$ its compile time, each under its own protocol — and pulls ahead as the machine grows, using $0.69\times$ its pairs on a 100–360-qubit QFT and QPEexact scale-up series. The two prove complementary rather than ranked, and taking the better of the two per instance uses 9.4% fewer pairs than DSABRE alone. Controlled experiments separate the contributions of capacity relief, the set of primitives, the initial layout and the horizon; relief earns its place on completion rather than cost, five of nine 80-qubit circuits failing outright without it. The implementation, harness and raw results are public.**

***Index Terms*—distributed quantum computing, quantum circuit compilation, multi-core quantum architectures, gate teleportation, cat-entangler packets, plan-oriented routing, entanglement cost**

I. INTRODUCTION

A distributed quantum computer (DQC) runs one circuit across several small cores joined by links that distribute entanglement on demand [1], [2]. Any two-qubit gate whose operands sit in different cores must be paid for with Einstein–Podolsky–Rosen (EPR) pairs, and distributing a pair is far slower and less reliable than any operation inside a core [3], [4]. Pairs consumed is therefore the metric we minimise (Section II-B).

A pair can be spent three ways. Teleporting a *state* (TELEDATA) relocates a logical qubit, making every later gate it meets in its new core local [5]. Teleporting a *gate* (TELEGATE) executes one straddling controlled gate through a cat-entangler, with neither operand moving [6], [7]. Holding that resource open across a run of straddling gates sharing a control costs one pair for the whole run — a *packet*, following AutoComm's amortisation [8]. Which is cheapest depends on the circuit structure where it is applied, so none dominates.

The choice must be made before its price is known. Each alternative becomes executable only after its own preparation — the right operand beside the right communication port, that port free, the destination holding a spare slot — and a compiler performs that work only for the alternative it selects. Scoring an action together with the preparation it currently needs confuses what remote work is worth doing with how far the layout is from doing it: candidates are ranked by the layout the compiler happens to occupy rather than by the progress they buy, and a router that reopens the choice mid-preparation can be left carrying SWAPs towards an action it abandons.

PORTER — a *Plan-Oriented Router for Teleportation-Enabled aRchitectures* — separates the two questions. Its unit of choice is a *plan*: a complete specification of one inter-core action and the conditions it needs, omitting the SWAP sequence that would establish them. Plans are compared on the gate progress they buy, one is committed, and only then are preparation SWAPs generated for it. Around this loop PORTER adds two-tier capacity relief, which manufactures free slots when every candidate is blocked, and a single deterministic initial layout in place of the best-of-N search its predecessors run.

Results. We evaluate on MQT-Bench [9] suites at 25, 64 and 80 logical qubits, a 100–360-qubit scale-up series, and two networks of IBM heavy-hex cores — forty-eight circuit–architecture instances. Against TELESABRE [10], the strongest published router we identified under this cost model, PORTER lowers geometric-mean EPR consumption by 59.7%, 44.9% and 68.8% on the three suites at four routes to its three, over the instances both complete, and it completes all twenty-four — including the seven TELESABRE does not converge on, among them an 80-qubit, 23,381-CX circuit.

The second comparison is with DSABRE [11], concurrent work on the same problem and the closest point of reference we have. PORTER is competitive on the three benchmark suites, spending $0.99\times$ its pooled EPR for $1.20\times$ its compile time under each compiler's own protocol, and pulls ahead as the machine grows: on the scale-up series it uses $0.69\times$ DSABRE's pairs, at $0.52\times$ its compile cost, and wins every instance.

A pooled figure that close invites the question of what a

F. Sheng, Y. Xie, and Y. Li are with the School of Mathematics and Statistics, Shaanxi Normal University, Xi'an, China; S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: sanjiang.li@uts.edu.au).

*ORCID: 0000-0002-3332-2546.

second compiler adds, and the per-instance record answers it: neither wins more than eleven of the twenty-four, and taking the better of the two per instance uses 9.4% fewer EPR pairs than DSABRE alone — 8.2% with the two cheapest circuit families set aside. A design that merely re-derived DSABRE's decisions would leave such a portfolio nothing to choose between. The two are close relatives and share four design decisions, set out in Section VII so neither is credited with the other's; what the portfolio measures is the part that is not shared.

Problem. Given a circuit, a multi-core machine and an initial layout, emit a legal operation stream realising every gate while minimising EPR pairs consumed. Section II-B states this formally, with the cost model it assumes and what that excludes.

Contributions. We make three.

(i) *The* PORTER *compiler*: a plan-oriented remote stage that scores a complete inter-core action before any preparation SWAP for it exists and commits before preparing (Section III-D), a capacity discipline that treats free slots as a legality condition and manufactures them by a two-tier relief search (Section III-D), and a deterministic initial layout replacing the best-of-N search its predecessors run (Section III-A).

(ii) *A lookahead-horizon portfolio, selected per circuit.* Every SABRE-descended router fixes this parameter by constant, and no tested constant is best across our benchmark — the best value spans a factor of thirty-two. PORTER routes every circuit at the same four horizons and keeps the lowest-EPR result. Nothing is conditioned on the circuit, so there is no threshold to generalise, and because the configured horizon is one of the four and wins ties, the portfolio cannot return a worse route than routing once. It is what carries PORTER past DSABRE at scale (Section IV-D1).

(iii) *A controlled study over forty-eight instances*, under native, shared-layout and cross-layout protocols, with ablations measuring what each mechanism contributes — including a negative finding, which Section VI interprets. Every route is certified by a separately implemented replay verifier, and the harness reproducing every table is released.

Section IV-A states the four questions the evaluation answers and ties each mechanism to the experiment that assesses it. The implementation, the replay verifier, the comparison harness and the raw per-circuit results are available at https://github.com/ebony72/porter.

This paper is self-contained. A supplement collects material that supports it without being needed to follow it: the reconstruction of the eighty-qubit benchmarks and the architecture and cat-entangler figures; the external `pytket-dqc` layouts, both as a comparison and as an opt-in layout candidate; the replay verifier's checks and a worked example of the plan-committed stage; the inter-core action mix, the per-circuit tables behind every summary reported here, and the full parameter set; a per-iteration complexity analysis; the layout-discount search we measured and did not adopt; and the threats to validity.

II. Model, Primitives, and Problem Formulation

A. Quantum Circuits and Routing

A circuit acts on n logical qubits in the basis $\{\texttt{id}, \texttt{rz}, \texttt{sx}, \texttt{x}, \texttt{cx}\}$ of MQT-Bench's Qiskit-opt3-transpiled circuits [9] — one-qubit gates plus CX — and is a directed acyclic graph with a dependency edge between consecutive gates on each logical qubit. The *front* is the set of gates with no unexecuted predecessor; only front gates can execute or be routed toward execution.

Physical qubits are partitioned into *cores*. Each core has an intra-core coupling graph, and pairs of cores may be joined by an inter-core link whose endpoints are *communication ports*. Every device here carries one link per adjacent pair and one port per neighbour, distinct across neighbours, so an adjacent pair fixes its port pair and a plan has no port to choose. A *layout* maps every logical qubit to a physical position. A two-qubit gate is *local* if both operands share a core and *remote* otherwise: a local gate that is not yet adjacent needs intra-core SWAPs, a remote gate needs an inter-core primitive. We write $\kappa_\ell(q)$ for the core holding q under layout ℓ, $d_{\mathrm{L}}(c, p, p')$ for intra-core distance and $d_{\mathrm{C}}(c, c')$ for core-graph distance.

B. Objective, Scope, and Assumptions

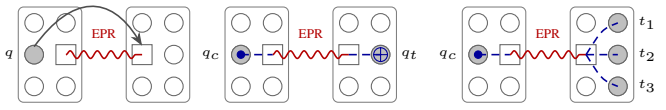
Objective. Given a circuit DAG C on n logical qubits, a machine $\mathcal{A}$ of K cores with intra-core coupling graphs, inter-core links and designated communication ports, and an initial layout ℓ_0, produce a legal operation stream that realises every gate of C while minimising the number of EPR pairs consumed. Intra-core SWAP count, circuit depth and compile time are recorded but are not part of the objective and are never traded against EPR pairs by any rule in this paper.

Cost model. One EPR pair is consumed per TELEDATA, per TELEGATE and per TELEGATEPACKET — one per packet, however many remote CX gates it amortises — charged the same whichever link carries it; a multi-hop relocation is a sequence of single-hop TELEDATA actions, charged once per hop. This is the model TELESABRE [10] and DSABRE [11] use, which is what makes the three compilers comparable in one unit. Communication ports are ordinary physical qubits, occupied for an action's duration and free otherwise; no core is assumed to have dedicated communication memory. What the model leaves out — link contention, entanglement latency and fidelity, purification, classical communication — is outside it, and every conclusion here is relative to that model. Appendix K collects this and the evaluation's other assumptions in one place.

C. Communication Primitives

Three inter-core primitives are in play, sketched in Figure 1; each consumes exactly one EPR pair.

State teleportation (TELEDATA). Consumes one EPR pair to move a logical qubit from a communication port in one core to a free port in another, after which every later gate that qubit meets in its new core is local. Its value is therefore *latent*: it is paid once and repaid over however many future gates the relocation makes local.



(a) TELEDATA (b) TELEGATE (c) TELEGATEPACKET

Fig. 1. The three communication primitives; each consumes one EPR pair between the two communication ports (squares). (a) TELEDATA moves qubit q (which must sit adjacent to its free port) onto the free destination port. (b) TELEGATE executes one remote CX through a cat-entangler [7] (Appendix E): the target-side port acts as a proxy for the control q_c; neither qubit moves. (c) TELEGATEPACKET brackets an ordered run of remote CX gates sharing the control q_c between one cat-entangler/disentangler pair, amortising a single EPR pair over the whole run; each target is routed next to the port in turn while the ports stay reserved.

Gate teleportation (TELEGATE). Executes a single remote CX with neither operand moving. One EPR pair entangles the two communication ports; a *cat-entangler* leaves (q_c, e_B) in the cat state $\alpha|00\rangle + \beta|11\rangle$, so every CX controlled by the remote port e_B acts as if controlled by q_c, and a *cat-disentangler* then restores q_c and frees both ports (Appendix E gives both constructions). Its value is entirely *direct*: exactly one gate, nothing left behind.

Same-control packets (TELEGATEPACKET). When several remote CX gates share the same control and their targets lie in one core, the run can be bracketed between a *single* cat-entangler and disentangler: the ports stay reserved, each target is routed beside e_B in turn, and the whole run costs one EPR pair instead of one each. This is the amortisation idea of AutoComm [8], specialised to the shared-control case and applied inside a routing loop rather than as a pre-pass.

The three differ in the respect that matters for scoring: how many gates one pair is spread over. TELEDATA amortises over an unknown future, TELEGATE over exactly one gate, TELEGATEPACKET over a run whose length the compiler must discover. Section III-E makes that a direct/latent/lookahead decomposition.

III. THE PORTER COMPILER

PORTER is a deterministic heuristic router for a circuit DAG C and an initial layout ℓ_0. At a fixed teleport-lookahead horizon L_{tp}, it repeatedly classifies the DAG front (Section III-B), executes all immediately executable work, routes local CX gates by intra-core SWAPs (Section III-C), and otherwise selects one inter-core *plan*, prepares it, and executes it (Algorithm 1). The plan-first stage separates the value of an inter-core action from the local SWAPs needed to make that action executable. The multi-horizon protocol that invokes this routing pass repeatedly is defined in Section IV-D1.

The implementation additionally bounds the total iterations and consecutive iterations that retire no gate, and detects repeated preparation states. These guards ensure termination of an implementation run; they are not a proof that every input is routed successfully (Section III-H).

A. Initial Layout

When no external layout is supplied, PORTER constructs one deterministic layout with at least $f_{\min} = 2$ initially free physical positions in each core. The same value has three distinct uses: it is the initial reserve, the admission threshold for a normal TELEDATA destination, and the free-slot threshold defining a relief reservoir. It is not a persistent lower bound: a teleport admitted at exactly $f_{\min}$ leaves its destination with one free position.

The layout heuristic is temporal locality-greedy. Each CX $g = (q_i, q_j)$ contributes $\gamma^{\text{asap}(g)}$ to an interaction weight $w(q_i, q_j)$, where $\gamma = 0.99$ and $\text{asap}(g)$ is the CX-only DAG layer of g. Logical qubits are visited in first-CX-occurrence order and assigned, when possible, to the core containing their heaviest already assigned neighbour while preserving the reserve. Within a core, more strongly interacting logical qubits receive more central physical positions. This favours early, repeated interactions without a best-of-N layout search. Its one weakness is that the reserve is uniform across cores, which suits a grid or a ring but not a core graph with a hub (Section V-E).

Algorithm 1 PORTER `compile`(C, ℓ_0)

```
ℓ ← ℓ0; initialise the active and front sets of the DAG
while the DAG has active gates and no implementation guard has fired do
    F ← current DAG front; classify its gates
    if a 1Q gate or a directly executable local CX is in F then
        execute all such gates; remove them from the DAG; continue
    end if
    if a local CX in F needs routing then
        apply the best intra-core SWAP under Eq. 1; continue
    end if
    generate normal plans P and blocked plans B
    score all plans and choose the first normal plan in key order
    if no normal plan is available then
        choose a tail-relief plan seeded by B
    else if B ≠ ∅ and its best member exceeds the selected normal plan by more than m_assist then
        replace the selection by an opportunistic-relief plan
    end if
    if no plan was selected then
        report capacity deadlock; abort
    end if
    prepare the selected plan by local SWAPs, then execute it
end while
return emitted operations, final layout, and statistics
```

B. Front Classification

Under layout ℓ, every front gate is one of the following:

- a 1Q gate, which is directly executable;
- a direct-local CX, whose operands are adjacent in one core;
- a local CX needing SWAPs, whose operands share a core but are not adjacent;

- an adjacent-remote CX, whose operand cores are neighbours; or
- a non-adjacent-remote CX.

PORTER drains directly executable gates first. It next routes local CX gates, and enters the remote stage only when every front CX is remote. This priority keeps ordinary intra-core routing separate from inter-core decisions.

C. Intra-Core SWAP Scoring

For the local-CX set F_{loc}, candidate SWAPs are coupling-graph edges incident to positions touched by F_{loc}. A candidate a is evaluated under its post-SWAP layout ℓ_a by the SABRE-style objective

$$J(a) = \bar{d}_{\mathcal{F}}(a) + \lambda_{\text{loc}} \bar{d}_{\mathcal{E}}(a), \qquad \lambda_{\text{loc}} = 0.8, \tag{1}$$

where the two terms are mean intra-core distances over F_{loc} and over up to $N_{\text{loc}} = 20$ additional local CX gates from the next $L_{\text{loc}} = 12$ DAG layers. The router applies the candidate with the smallest J, breaking ties by front-layer distance and then deterministically.

D. Plans and the Plan-First Remote Stage

A *plan* is one candidate inter-core action across a single link. It records its primitive, its source and destination cores, the link and ports it uses, the gates it discharges, and the front CX it was generated from — for a relief plan, the CX behind the blocked plan it serves rather than one the action itself advances. It deliberately does *not* record a preparation-SWAP sequence. Thus plans answer "what remote action should be taken next?"; preparation answers "how do we make that action executable from the present layout?" (Section III-F).

The primitive is one of $\{\text{TELEDATA}, \text{TELEGATE}, \text{TELEGATEPACKET}\}$. A TELEDATA plan moves one logical qubit by one core-graph hop and immediately discharges no CX. A TELEGATE plan discharges one adjacent-remote CX. A TELEGATEPACKET plan discharges an ordered, same-control sequence of adjacent-remote CX gates (Section III-G). For the gate-teleportation family, the plan's source and destination denote the control-side and target-side cores; for TELEDATA they denote the sending and receiving cores.

Three kinds of plan. Every plan the router handles is *normal*, *blocked* or *relief*. The three share the record above, the score of Section III-E and the readiness predicate of Section III-F; they differ in how they are generated and in what the router is permitted to do with them.

Normal plans $\mathcal{P}$ are the actions that could fire once prepared, generated from the remote front. For each adjacent-remote front CX, $\mathcal{P}$ contains a TELEGATE plan, a packet plan when a packet of at least two gates is available, and each direction of TELEDATA whose receiving core has at least $f_{\min}$ free positions. For a non-adjacent-remote CX it contains only one-hop TELEDATA moves that strictly reduce the distance between operand cores and meet the same capacity test. PORTER commits to the best of them.

Blocked plans $\mathcal{B}$ are the TELEDATA moves that would otherwise be generated but whose destination fails that capacity test. They are never executed — committing one would drive a core below the floor — but they are generated and scored, because $\mathcal{B}$ is the router's measurement of the work capacity is currently preventing: it names the saturated cores and says how much gain is being forgone at each.

Relief plans are generated on demand, to create space for a blocked one. Starting at the congested destination core c, a breadth-first search finds a nearest *reservoir*, a core with at least $f_{\min}$ free positions; the relief plan is the TELEDATA moving one logical qubit from the penultimate core of that path into the reservoir. Only that final hop is committed, and after it executes the router regenerates every plan, so a multi-hop route propagates free capacity one hop at a time rather than reserving an atomic relay (Section VII). A relief plan is always newly generated for the purpose; the blocked plan that motivated it is never itself committed.

When each fires. Normally the best member of $\mathcal{P}$ is committed. Two rules override that, both driven by the best-scoring member of $\mathcal{B}$ alone. *Tail relief* fires reactively, when $\mathcal{P}$ is empty and the front is starved. *Opportunistic relief* fires pre-emptively, while a normal plan is still available, when the best blocked plan out-scores it by more than $m_{\text{assist}} = 5.0$ — the judgement being that clearing the congestion now is worth more than taking the best currently legal action. Both tiers run the same reservoir search and commit one hop of it, and scoring relief on the same key as ordinary work is what lets the opportunistic tier compete with it.

E. Plan Scoring and the Selection Key

Every candidate receives the lexicographic key

$$\text{key}(\mathsf{p}) = \big(-\text{total}(\mathsf{p}),\ \text{prep}(\mathsf{p}),\ -f_{\text{dst}}(\mathsf{p})\big), \tag{2}$$

ordered from smallest to largest, with a stable identifier as the final tie break. Here f_{dst} is the free capacity of a TELEDATA destination and is defined to be zero for the TELEGATE family. Hence the primary comparison is always total; readiness cost matters only when utility ties, and capacity is a final tie break. In particular, it also implements a TELEDATA-first preference when a teleport and a gate plan tie on total, because an admissible teleport has $f_{\text{dst}} \geq f_{\min}$.

total is a ranking heuristic, not a common physical unit: gate teleportation counts CX gates discharged immediately, whereas a data teleport is valued by current and predicted reductions in core distance. Specifically, a TELEGATE has $\text{total} = 1$, and a packet has total equal to the number of CX gates in its collected sequence.

For a TELEDATA plan moving q from c_{src} to c_{dst}, let κ and κ' map logical qubits to cores before and after that one-hop move. The progress of a CX $g = (q_i, q_j)$ is

$$\Delta(g) = d_{\text{C}}\big(\kappa(q_i), \kappa(q_j)\big) - d_{\text{C}}\big(\kappa'(q_i), \kappa'(q_j)\big). \tag{3}$$

Thus $\Delta(g) \in \{-1, 0, 1\}$. Forward simulation under κ' forms three disjoint sets of CX gates incident on q: $D(\mathsf{p})$ contains the seed front CX; $L(\mathsf{p})$ contains CXs exposed while locally executable gates are repeatedly retired; and $A_k(\mathsf{p})$ contains gates reached after that local closure in simulated layer k, for $1 \leq k \leq L_{\text{tp}}$. The first two describe

immediate/local-closure progress; the third is lookahead. With $w_k = \max(w_{\min}, w_0\gamma^{k-1})$, $w_0 = 0.9$, $\gamma = 0.9$, and $w_{\min} = 0.5$,

$$\begin{aligned} \mathrm{direct}(\mathsf{p}) &= \sum_{g \in D(\mathsf{p}) \cup L(\mathsf{p})} \Delta(g), \qquad (4) \\ \mathrm{lookahead}(\mathsf{p}) &= \sum_{k=1}^{L_{\mathrm{tp}}} \sum_{g \in A_k(\mathsf{p})} w_k \Delta(g), \\ \mathrm{total}(\mathsf{p}) &= \mathrm{direct}(\mathsf{p}) + \mathrm{lookahead}(\mathsf{p}). \end{aligned}$$

The best member of $\mathcal{P}$ under key (2) is committed, unless one of the two relief rules of Section III-D overrides that choice.

F. Commitment and Preparation

A plan is *ready* in layout ℓ, written $\mathrm{ready}(\mathsf{p}, \ell)$, when the required logical endpoint(s) are adjacent to their designated communication ports, the required ports are available, and, for TELEDATA, the destination satisfies the capacity rule. The readiness heuristic $\mathrm{prep}(\mathsf{p}, \ell)$ is the sum of the intra-core distances from required endpoints (or the destination free slot) to their required ports. It is zero exactly for a ready plan. It is a tie-breaking estimate of remaining local routing work, not part of plan utility or a claimed exact SWAP count.

Because plans are scored before any plan-specific preparation SWAP exists, the choice is made on what an action is worth rather than on what it would cost to set up. PORTER commits to the selected plan, performs only local SWAPs intended to reduce its readiness cost, and executes the plan once it is ready. It does not reopen the remote choice during that preparation. Commitment therefore prevents local moves made for one remote action from being reinterpreted as preparation for a different action.

For an unmet endpoint requirement, preparation considers incident intra-core SWAPs. For an occupied TELEDATA destination port, it considers SWAPs that move a nearest free position toward that port without swapping through the port prematurely. Candidates are ranked by post-SWAP prep, and at most one move per plan side is applied in a preparation round.

Preparation always completes. Every core retains a free position, a teleport entering a destination only when it has at least $f_{\min} = 2$ and therefore leaving one, so on a connected core that free position can be walked to an occupied port to clear it, and the endpoint then walked to a neighbour of that port along a path meeting no edge incident to it — at most a core diameter of local SWAPs apiece. The two sides lie in different cores and a local SWAP moves no qubit between cores, so neither side can undo the other and the capacity rule survives preparation.

G. Same-Control Packet Collection

A packet is discovered by forward simulation because CX gates sharing a control are generally chained in the DAG and therefore do not appear in the front together. From an adjacent-remote seed $g = (q_c, q_t)$ whose target lies in core c_t, the collector simulates up to 120 layers. It absorbs up to eight front CX gates with control q_c and target in c_t, together with executable 1Q gates already in c_t, and stops at the first unproductive layer or the size cap. A packet is generated only if at least two CX gates are collected.

Packet readiness means readiness to establish the cat resource, not readiness of every later target. Once committed, the router emits one `cat_entangler`, routes each recorded target beside the target-side port without disturbing either reserved port, emits its `cat_cx`, and finally emits `cat_disentangler`. The recorded order follows the DAG, so the packet plan may safely count all of its members as discharged work.

H. Progress and the Failure Model

The capacity discipline prevents routing from creating a completely full core: every normal or relief TELEDATA is admitted only when its destination has at least two free positions, and therefore leaves at least one. A supplied layout with a full core is first legalised by an eviction if another core can receive it; otherwise the input is infeasible.

This invariant does not prove universal successful routing. If normal plans are absent but blocked plans identify a reachable reservoir, tail relief creates a new candidate. If no such candidate exists, PORTER reports a capacity deadlock. Independently, the implementation bounds every loop — repeated preparation states, an iteration budget and a no-progress limit — safeguards on the greedy realisation of the policy rather than conditions the argument of Section III-D needs. Thus every emitted route is checked for legality, and every implementation run terminates, but successful completion on every feasible input is not claimed — the guarantee DSABRE has and PORTER does not (Section VII). A per-iteration complexity analysis is given in Appendix I.

IV. EXPERIMENTAL METHODOLOGY

This section specifies the evaluation questions, experimental inputs, and comparison protocols. A *route* is one invocation of the router on one circuit from one initial layout. A *protocol* specifies how layouts are obtained, how many routes are run, and how the reported route is selected. Reported compile time is the cost of the complete protocol, including layout generation and all unsuccessful candidate routes.

A. Research Questions

The experiments address four questions.

RQ1 Under each compiler's native protocol, how do EPR-pair consumption, completion, and complete compile time compare?

RQ2 When the initial layout is controlled, how much of the difference between PORTER and DSABRE is due to routing rather than layout generation?

RQ3 What are the individual effects of capacity relief, the communication primitives, initial layout, and horizon selection?

RQ4 Do these conclusions persist as circuit width, core count, core shape, and core-graph topology change?

Section V follows this order. Each results subsection states the configuration held fixed and its metric before interpreting the outcome.

B. Experimental Setup

Implementation and checking. PORTER is implemented in Python and released with the comparison harness. It is deterministic: ties follow a fixed key order and any residual tie uses a constant-initialised generator. Thus, a fixed circuit, machine, layout, and horizon produce a byte-identical operation stream. A separately implemented replay verifier checks every reported PORTER stream for gate coverage, dependency order, and physical legality; the verifier is tested by fault injection (Appendix D).

Architectures. We use the two grid-of-grids devices evaluated by DSABRE [11]: architecture A, a 2×2 array of 3×3 cores (36 physical qubits), and architecture H, a 2×3 array of 4×4 cores (96 physical qubits). RQ4 also uses DSABRE's 5×5-core scale-up series and two networks of IBM 27-qubit heavy-hex tiles. Device descriptions are read from the baselines' own files so that all compilers use identical physical-qubit identifiers.

Benchmarks. The main inputs are the MQT-Bench files used in the DSABRE evaluation [9], verified by exact CX-count match. Architecture A uses six 25-qubit families (AE, GHZ, Graphstate, QFT, QNN, and Random). Architecture H uses nine 64-qubit families, adding QPEexact, QAOA, and Multiplier. We also construct an 80-qubit suite of the same nine families; its AE and QNN instances are reconstructed because the stock generator does not produce them. The supplement documents these reconstructions (Appendix A).

Baselines and execution. We compare with TELESABRE [10] and DSABRE [11]. DSABRE uses its shipped safe mode and layout rule. Its recovery budget is derived from the architecture as in its drivers, and its pre-routing layout-repair EPR count is excluded for both compilers because it changes the starting layout rather than routing the circuit. The stock TELESABRE binary aborts reproducibly in its initial-layout pass whenever no core holds more free positions than its configured margin, which is every 25- and 80-qubit instance and no 64-qubit one. We therefore use and release a minimal patch that assigns the logical qubits the stock code leaves unplaced to the least-loaded core and changes nothing else, so results agree with the stock binary wherever it completes — the whole 64-qubit suite. Each compiler invocation has a 900-s wall-clock limit; an invocation that exceeds it is recorded as incomplete.

C. Layouts and Comparison Protocols

We use native, shared-layout, and cross-layout comparisons. These controls separate the quality of an initial layout from the quality of the routing loop, but they do not make all resource budgets identical.

Native protocols (RQ1 and RQ4). TELESABRE uses its Hungarian layout heuristic at seeds 0–2 and reports the best EPR result. DSABRE uses Qiskit `SabreLayout` at seeds 0–2, followed for each candidate by its forward–backward–forward routing schedule, and reports the best result. PORTER builds one deterministic temporal-locality-greedy layout (Section III-A), routes it at four horizons, and retains the lowest-EPR route. PORTER+R additionally performs backward refinement from that selected route.

For DSABRE, `SabreLayout` operates on its shipped corners-removed coupling map. The removed low-degree corner positions begin empty and provide capacity for arrivals and evictions; they are distinct from the designated communication ports.

Shared layouts (RQ2). Both DSABRE and PORTER route the same externally supplied layout. We use TELESABRE's layout where it is available and, separately, the `pytket-dqc` hypergraph partition (Appendix B). This controls the layout but not the routing effort: PORTER evaluates four horizons, whereas DSABRE uses three passes.

Cross layouts (RQ2). Each compiler also routes the other compiler's native layout(s). In particular, PORTER routes all three of DSABRE's candidate layouts and selects the lowest-EPR outcome. This test shows whether a native-protocol result is sensitive to the layout source.

D. Operating Protocols

Section III defines one routing pass. The following protocols repeat that pass; they do not change the router itself.

1) PORTER*: fixed horizon portfolio:* The teleport-lookahead horizon L_{tp} controls how far a TELEDATA candidate is evaluated into the DAG. No single value was best in an offline sweep over $\{4, 8, \ldots, 1024\}$: the per-instance optimum spans the range. For example, 80-qubit QNN selects 16, whereas every 100-, 200-, and 360-qubit scale-up instance selects 128. We therefore use the same four-value portfolio for every circuit,

$$L_{\mathrm{tp}} \in \{4, 8, 16, 128\},$$

and retain the route with the fewest EPR pairs. The configured default, $L_{\mathrm{tp}} = 16$, runs first and wins ties. Consequently, provided that the default route completes, the portfolio cannot report more EPR pairs than that one-route baseline.

We deliberately do not select a horizon from a circuit statistic: a predictive rule would be fitted and evaluated on the same instances, and the portfolio needs no such rule, since the configured horizon is one of its candidates. The portfolio itself was also chosen after inspecting the 48 evaluated instances, so its measured EPR gain is in-sample rather than a held-out generalisation result. The portfolio is robust in the limited sense that nearby four-value sets differ by less than one percent in the reported sweep, but out-of-sample validation remains necessary. Its cost is additional compile time, especially on dense circuits where a long lookahead is expensive.

2) PORTER+*R: backward refinement:* PORTER+R applies SABRE-style bidirectional refinement [12] to complete routes. Let $R(C, \ell)$ denote routing circuit C from layout ℓ. Pass 1 routes C from ℓ_0. Pass 2 routes the reversed circuit from the final layout of pass 1 and retains only its final layout. Pass 3 routes C from that layout. We use five passes by repeating this forward–backward cycle and report the best forward route.

The first pass is the four-horizon PORTER portfolio. Its selected horizon is retained for the remaining four passes, rather than being reselected at each pass. Thus PORTER+R executes eight routes in total and contains the PORTER result; retaining the best forward route ensures that it cannot report more EPR pairs than PORTER. Individual refinement passes can still be worse because a post-routing layout need not preserve the free-slot reserve of the original layout.

E. Budgets and Metrics

Table I makes the search budget explicit. "Routes" counts all routing invocations, including horizon probes. Thus route count is a transparent effort descriptor, not a claim that routes have equal runtime.

The primary outcome is EPR pairs consumed. TELESABRE counts TELEDATA and TELEGATE operations; DSABRE counts TELEDATA operations; and PORTER counts TELEDATA, TELEGATE, and TELEGATEPACKET operations, one pair per packet. Under the cost model of Section II-B, these are the same physical resource. We also report intra-core SWAP counts and wall-clock compile time, but no selection rule trades either against EPR count.

Aggregate EPR results are unweighted geometric means over circuits. For a comparison of methods a and b, the reported ratio is b/a and the percentage change is $100(b/a-1)$, calculated from the aggregate means; a negative percentage therefore means that b uses fewer pairs than a. Failures are not imputed. Each paired mean includes only circuits completed by all columns in that comparison, and every such mean reports its completion count. This convention may make a failing baseline appear better by excluding hard instances; completion counts are therefore essential context rather than a secondary outcome.

Compile times are complete protocol wall-clock times measured once per configuration on an otherwise idle Apple M2 system (macOS 26.5, CPython 3.13), without warm-up. They should be interpreted as indicative protocol-level ratios, not as portable absolute timings.

V. RESULTS

Each subsection answers one research question of Section IV-A. Protocol names and budgets are those of Table I; lower EPR is better throughout.

A. RQ1: End-to-End Cost Under Each Compiler's Own Protocol

Question: how do the three compilers compare when each uses its own recommended protocol? *Held fixed:* nothing — each compiler supplies its own layouts and pass schedule (Table I). *Metric:* geometric-mean EPR over jointly completed circuits, plus completion counts and complete compile time.

Table II summarises the three suites; Appendix H gives the per-circuit EPR and SWAP counts. Baseline counts come from each baseline's own harness, and on the eight 64-qubit circuits TELESABRE completes ours reproduces the counts DSABRE publishes for it [11] exactly.

Against TELESABRE the ordering is consistent across the suite. On block *(a)*, from one deterministic layout, PORTER lowers geometric-mean EPR by 59.7%, 44.9% and 68.8% at 25, 64 and 80 qubits, spending four routes against TELESABRE's three seeded runs, and every circuit TELESABRE completes improves, from -23.2% to -81.0%. Refinement adds to that at 25 and 64 qubits (63.5% and 46.9%) but little at 80 (70.3%), where the horizon rather than the extra routes does the work. TELESABRE completes seventeen instances, failing on 64q Random and six of the nine 80-qubit circuits; PORTER and DSABRE complete all twenty-four.

Against DSABRE the answer varies by suite, and block *(b)* is the basis since both complete everything. At four routes to DSABRE's nine, PORTER takes the geometric mean at 25 qubits ($0.94\times$) and at 80 ($0.93\times$) and trails at 64 (186.1 against 173.1, $1.07\times$), winning eleven instances, losing ten and tying three; pooled the two are level, 169.4 against 171.9 ($0.99\times$). Refinement extends that at eight routes to 156.7 ($0.91\times$) and narrows 64 qubits to $1.04\times$, so the one suite PORTER loses under its own protocol is also the one extra routes largely close. PORTER+R's largest saving is 80q Multiplier, $2{,}130$ against $4{,}841$; its worst loss 80q Random, $10{,}848$ against $10{,}551$.

B. RQ2: Routing Quality With the Layout Controlled

Question: with the initial layout held identical, does either routing loop use fewer EPR pairs? *Held fixed:* the layout, taken in turn from each of the three sources available. *Metric:* geometric-mean EPR over all twenty-four instances.

The three sources are those that place all twenty-four circuits. TELESABRE is not among them: it converges on seventeen and emits a layout only where it converges (Section V-A).

Changing the routing loop moves less than changing the layout. Across any row of Table III, from the same layout, the two land within 1.9% of one another on block *(a)* and within 6.5% on *(b)*, PORTER ahead in two rows of three in each block and DSABRE taking `pytket-dqc`'s partition. The layout is controlled but the budget is not: PORTER spends four routes on its candidate horizons to DSABRE's three passes, so a narrow PORTER lead here is not a like-for-like win. On the eighteen non-trivial instances the column spread is 9.2–14.5% against 0.9–6.1% across a row, so the initial layout is the larger source of variation for both compilers; pooled over all twenty-four the two are comparable, so that reading is a claim about the non-trivial circuits.

Which layout is best is not, and PORTER does not adopt the partition. Over all twenty-four instances `pytket-dqc`'s hypergraph partition is the lowest cell in every column; drop GHZ and Graphstate and it is the highest in every column, raising the mean by 10.7% for DSABRE and 14.5% for PORTER. Those six rows carry the whole result, and the partition is the worse start on fourteen of the twenty-four instances for DSABRE and sixteen for PORTER, at a median ratio of 1.03 and 1.07. Its gain is real but confined to the two structurally simplest families, whose interaction graphs a partitioner cuts almost perfectly and whose absolute cost is

TABLE I

CONFIGURATIONS USED IN SECTION V: LAYOUT SOURCE, LAYOUT COUNT $|\mathcal{L}|$, ROUTES SPENT PER LAYOUT P, TOTAL ROUTES, AND WHAT IS SELECTED. A ROUTE IS ONE ROUTER INVOCATION. P IS THE FOUR CANDIDATE HORIZONS ON THE ROWS THAT SELECT A HORIZON AND NOTHING ELSE, AND THE FIVE REFINEMENT PASSES ON THE +R ROWS, WHOSE FIRST PASS IS ITSELF THE FOUR-HORIZON PORTFOLIO — HENCE $4 + 4 = 8$.

Configuration	Layout source	$\|\mathcal{L}\|$	P	Routes	Selection rule
Native protocol (RQ1, RQ4)					
TELESABRE	Hungarian, seeds 0–2	3	1	3	lowest EPR
DSABRE	SabreLayout, seeds 0–2	3	3	9	lowest EPR
PORTER	temporal locality-greedy	1	4	4	lowest EPR
PORTER+R	temporal locality-greedy	1	5	8	best forward route
Shared layout (RQ2)					
DSABRE shared	one external layout	1	3	3	best pass
PORTER shared	the identical layout	1	4	4	lowest EPR
PORTER+R shared	the identical layout	1	5	8	best forward route
Cross layout (RQ2)					
DSABRE on PORTER	PORTER layout	1	3	3	best pass
PORTER on DSABRE	three DSABRE layouts	3	4	12	lowest EPR
Ablation (RQ3)					
PORTER ablation	temporal locality-greedy	1	1	1	—

TABLE II

MAIN COMPARISON, OWN PROTOCOL: EACH COMPILER FROM ITS OWN INITIAL LAYOUT AND ROUTE BUDGET (TABLE I). BLOCK *(a)* AVERAGES THE CIRCUITS *all four* COMPLETE, THE ONLY BASIS ON WHICH TELESABRE CAN BE COMPARED; BLOCK *(b)* AVERAGES EVERY CIRCUIT AND DROPS IT. THE BLOCKS DIFFER ONLY IN WHICH CIRCUITS THEY AVERAGE. "DONE" IS CIRCUITS COMPLETED AND "W/L/T" THE PER-INSTANCE RECORD AGAINST DSABRE OVER ALL TWENTY-FOUR; t IS COMPLETE PROTOCOL TIME (GMEAN, SECONDS) OVER THE WHOLE SUITE. THE LAST PAIR IS THE PORTFOLIO†, A DERIVED QUANTITY RATHER THAN A FIFTH CONFIGURATION, SO IT IS LEFT UNBOLD. BOLD MARKS THE LOWEST EPR COUNT OF THE FOUR COMPILERS IN EACH ROW; TIMES ARE NEVER BOLD. LOWER IS BETTER; PER-CIRCUIT COUNTS IN APPENDIX H.

	TELESABRE		DSABRE		PORTER		PORTER+R		best of DSABRE with†	
Suite	EPR	t	EPR	t	EPR	t	EPR	t	PORTER	+R
(a) over the circuits *all four* complete										
25q (6)	106.6	0.51	45.8	0.23	43.0	0.21	**38.9**	0.36	41.9	38.9
64q (8)	284.5	11.89	**144.6**	1.82	156.8	2.24	151.1	3.93	140.5	139.6
80q (3)	121.0	5.11	44.9	3.59	37.8	5.00	**35.9**	9.35	33.8	33.8
(b) over *every* circuit in the suite; TELESABRE omitted										
25q (6)	—	—	45.8	0.23	43.0	0.21	**38.9**	0.36	41.9	38.9
64q (9)	—	—	**173.1**	1.82	186.1	2.24	179.6	3.93	168.8	167.4
80q (9)	—	—	412.1	3.59	384.7	5.00	**346.1**	9.35	344.7	333.4
all (24)	—	—	171.9	1.40	169.4	1.68	**156.7**	3.00	155.7	150.5
done	17 / 24		24 / 24		24 / 24		24 / 24		—	
W/L/T	—		—		11 / 10 / 3		14 / 8 / 2		—	

† **Portfolio.** For each instance, the lower of DSABRE's EPR count and the named PORTER configuration's, each compiler under its own protocol and its own initial layout: no layout is shared here, and neither router is given the other's mapping — for that control see Table III. The cell is the geometric mean of those per-instance minima over the row's circuits, which is why it can fall below *both* compilers' means: they lose on different circuits. At 64 qubits DSABRE is cheaper on six of the nine and PORTER on two, and PORTER's 1,076 against 1,285 on Multiplier is what carries the pair to 168.8, under DSABRE's own 173.1. Realising it costs both compile budgets, and no rule we tested predicts the per-instance winner in advance (Section VI).

least. It is also expensive: best-of-three KaHyPar seeds costs 0.6–627 s against 0.3–9.0 ms for PORTER's construction, on most circuits more than PORTER's whole four-route protocol, and it does not finish at the widths of Section V-D. Cost and confinement are why PORTER keeps its own layout and treats the partition as an opt-in candidate (Appendix C). Neither effect is specific to PORTER — the same partitions are the worst start for DSABRE's loop on the same eighteen instances — plausibly because `pytket-dqc` optimises e-bits on a network more permissive than the device [11].

Refinement pays from every starting layout, once the horizon is free to move. PORTER+R gains 8.2% over PORTER on DSABRE's layout (170.3 → 156.3), 8.0% on `pytket-dqc`'s (167.2 → 153.8) and 7.5% on its own (169.4 → 156.7), overtaking DSABRE's own protocol from all three at eight routes to its nine.

C. RQ3: Mechanism Ablation

Question: what does each mechanism contribute on its own? *Held fixed:* the initial layout, the lookahead horizon and every other parameter; exactly one mechanism is disabled per column. *Metric:* EPR and completion.

Table IV disables each mechanism on the 80-qubit suite, holding layout, horizon and every other parameter fixed; Appendix G gives the per-action counts. Every column is one

TABLE III
LAYOUT AGAINST ROUTER: GEOMETRIC-MEAN EPR FOR EVERY COMBINATION OF INITIAL LAYOUT (ROWS) AND ROUTING LOOP (COLUMNS). PORTER SPENDS FOUR ROUTES PER LAYOUT TO DSABRE'S THREE PASSES, PORTER+R EIGHT OVER FIVE PASSES. BLOCK *(a)* IS OVER ALL TWENTY-FOUR INSTANCES; BLOCK *(b)* REPEATS IT WITHOUT GHZ AND GRAPHSTATE, WHOSE COUNTS ARE ONE TO TWO ORDERS OF MAGNITUDE BELOW THE REST AND SO DOMINATE AN UNWEIGHTED GEOMETRIC MEAN. THE BLOCKS DISAGREE IN EVERY COLUMN, WHICH IS THE TEXT'S SUBJECT. LOWER IS BETTER; BEST IN EACH COLUMN OF EACH BLOCK BOLD.

Initial layout	DSABRE	PORTER	PORTER+R
(a) all twenty-four instances			
DSABRE's `SabreLayout`	171.9	170.3	156.3
PORTER's temporal locality-greedy	171.2	169.4	156.7
`pytket-dqc`'s partition	**164.1**	**167.2**	**153.8**
(b) the eighteen excluding GHZ and Graphstate			
DSABRE's `SabreLayout`	**452.5**	448.7	426.2
PORTER's temporal locality-greedy	473.9	**445.2**	**422.7**
`pytket-dqc`'s partition	501.1	509.9	461.8

TABLE IV
MECHANISM ABLATION, 80-QUBIT SUITE: PORTER EPR WITH ONE DESIGN DECISION DISABLED, FROM THE SAME LAYOUT AT THE SAME HORIZON. "NO PK" DISABLES PACKETS, "NO REL." BOTH RELIEF TIERS, "TELEDATA" THE TELEGATE FAMILY; "PK"/"TG" ARE THE FULL CONFIGURATION'S PACKET AND SINGLE-TELEGATE COUNTS. ALL NINE CIRCUITS ARE SHOWN; GHZ AND GRAPHSTATE COINCIDE IN EVERY COLUMN, NO MECHANISM BEING EXERCISED ON THEM. THE GMEAN LINES' n DIFFERS BECAUSE THE COLUMNS FAIL ON DIFFERENT CIRCUITS. †DOES NOT COMPLETE. ‡NEEDS THE EXTRA RELIEF TIER.

Circuit	Full	No pk	No rel.	TELEDATA	pk	tg
ae	331	367	†	342‡	54	1
ghz	7	7	7	7	0	0
graphstate	28	28	28	28	0	0
qft	318	337	†	†	78	0
qnn	1072	1099	†	1132‡	113	2
random	11245	†	†	†	537	3792
qpeexact	435	443	†	**347**‡	124	0
qaoa	1202	1146	1505	†	360	21
multiplier	3897	3584	3055	†	208	1739
gmean, n=8 of 9	**286.2**	288.9				
gmean, n=4 of 9	174.1		**173.3**			
gmean, n=5 of 9	124.8			**121.4**		

forward pass at the configured horizon, so Full is that pass, not Table II's selected route. What each column asks is whether its mechanism earns its place.

Packets. Disabling TELEGATEPACKET makes every shared-control remote CX pay per gate. At 80 qubits that costs only 0.9% over the eight rows both columns complete, and the sign splits: AE pays +11%, QFT +6%, QNN +3% and QPEexact +2%, while QAOA is 5% and Multiplier 8% *cheaper* without packets — the two packet-heaviest rows in the table. The primitive still pays at the smaller widths (3.0% at 25 qubits, 7.4% at 64), so batching stops earning its keep where the front is widest and a packet's forward simulation displaces a better teleport plan. Read alone the column is near neutral; the next explains why.

TELEDATA only. This column is two configurations. *Strict removal* of the TELEGATE family is the one-factor ablation, and it fails on all but two rows: the TELEGATE is the only remote action needing no free *data* slot, since its cat copy occupies the target-side communication qubit instead of displacing a logical qubit, so it alone is not gated on $\mathrm{free}(c) \geq f_{\min}$. Without it a front of adjacent remote CXs whose destinations all sit at $f_{\min}$ has an empty plan set. The TELEGATE earns its place on *completion*, then, not on termination, which the guards of Section III-H give either way.

The vocabulary variant (‡) also feeds adjacent-blocked candidates to the relief BFS, recovering five of the seven; QFT, Random, QAOA and Multiplier still do not complete. That tier leaves the full configuration EPR-identical, so what it isolates is vocabulary and not relief — though it is no longer a strict ablation. On that basis teleport-only is 2.7% cheaper at 80 qubits, carried by one row — QPEexact at 347 against 435 — while it is *dearer* on AE (342 against 331) and QNN (1,132 against 1,072), so the ordering TELEDATA only < Full < No packets holds on one of the five rows where all three complete. Pooled over the nineteen it completes it is 3.5% dearer, on the same width trend as the packets column (+2.2%, +8.6%, −2.7%): teleports alone are competitive only at the widest circuits, and beat the full set outright nowhere. Section VI reads the residue as a currency mismatch in the plan comparator, not a verdict on packets. It also bears on DSABRE's open question: it chose teledata-only by measurement, leaving grouping untested, and grouping pays at the widths where its own suite sits.

Capacity relief. Disabling both tiers costs completion, not EPR pairs. Five of the nine circuits fail outright: with every destination below $f_{\min}$ and no way to drain qubits toward reservoirs, no legal remote plan remains. Where routing completes it is 0.5% *dearer* than doing without, and Multiplier is cheaper without it, so the opportunistic tier can overpay where congestion would have cleared itself: at this width relief earns its place on completion alone, though pooled over the nineteen that complete both ways it still saves 24.8%. With both tiers on no instance here reached the deadlock path of Section III-H; with both off, five did — a statement about these instances, not a guarantee.

D. RQ4: Scale-Up to 360 Logical Qubits

Question: do the RQ1 and RQ2 answers hold as circuit width and core count grow together? *Held fixed:* DSABRE's own scalability architectures and circuit files. *Metric:* geometric-mean EPR and complete compile time.

Every result so far is at 108 physical qubits or fewer. Table V takes the two circuit families that exist at 100, 200 and 360 logical qubits onto DSABRE's scalability series: 5×5 cores throughout, core count 6 → 12 → 20, so occupancy stays roughly constant while the core graph grows and inter-core traffic with it — DSABRE's own cost rises 200 → 765 → 1,890 on QFT.

PORTER uses fewer EPR pairs than DSABRE on all six instances, by a widening margin: 0.90× at 100 logical qubits, 0.66× at 200, 0.57× at 360, for 0.69× overall. It does so at 0.52× DSABRE's complete compile cost (8.0 s against 15.4 s), routing four times where DSABRE routes nine.

The horizon is what makes that difference, and the comparison is available directly. Routing the same six instances

TABLE V
SCALE-UP SUITE: QFT AND QPEEXACT AT 100, 200 AND 360 LOGICAL QUBITS, EPR PAIRS, EACH COMPILER UNDER ITS OWN LAYOUT AND PROTOCOL. ARCHITECTURES FOLLOW DSABRE'S SCALABILITY PROTOCOL (5×5 CORES, N_{phys} PHYSICAL QUBITS IN $K = 6 \to 12 \to 20$ CORES), READ FROM ITS OWN DEVICE FILES; OUR DSABRE COLUMN REPRODUCES ITS PUBLISHED SERIES [11] (201/246/766/896/1890/1938). "t" IS COMPLETE PROTOCOL TIME: NINE ROUTES FOR DSABRE, FOUR FOR PORTER, WHICH SELECTS L_{tp}=128 ON ALL SIX. `PYTKET-DQC` IS OMITTED — ITS PARTITIONER DOES NOT FINISH AT THESE WIDTHS. LOWER IS BETTER.

		architecture			DSABRE		PORTER	
Circuit	CX	n	N_{phys}	K	EPR	t	EPR	t
QFT	3,420	100	150	6	200	3.6	**198**	3.0
QPEexact	3,579	100	150	6	246	4.3	**200**	2.4
QFT	7,220	200	300	12	765	13.9	**564**	9.3
QPEexact	7,579	200	300	12	896	13.9	**528**	9.0
QFT	13,300	360	500	20	1890	53.9	**1074**	18.6
QPEexact	13,979	360	500	20	1938	83.1	**1093**	22.5
gmean (6)					705.7	15.4	**490.0**	**8.0**

TABLE VI
THE NINE-CIRCUIT 64-QUBIT SUITE ON FOUR IBM 27-QUBIT HEAVY-HEX CORES (108 PHYSICAL), WIRED AS A *ring* (ALL CORES DEGREE 2) AND A *star* (CORE 0 A HUB EVERY ROUTE CROSSES). GEOMETRIC-MEAN EPR; BOTH COMPILERS COMPLETE ALL EIGHTEEN INSTANCES. THE LAST COLUMN IS THE FIXED-LAYOUT CONTROL — PORTER ROUTING DSABRE'S OWN `SABRELAYOUT` CANDIDATES, SO ONLY THE ROUTING LOOP DIFFERS. PER-CIRCUIT ROWS ARE IN APPENDIX H.

		PORTER	
Core graph	DSABRE	own layout	DSABRE's layout
ring (9)	85.2	86.1	**82.9**
star (9)	101.1	113.3	**97.2**

from the same layouts at the configured horizon alone — one route, no selection — gives a geometric mean of 1,026.5 pairs against DSABRE's 705.7, which is 1.45× and loses all six. Selection changes nothing else about the compiler and reaches 490.0, which is 0.69× and wins all six. Every instance selects L_{tp}=128, the longest candidate offered, and none selects the configured 16: at these widths a plan's value is realised further into the circuit than the default horizon can see.

E. RQ4: Heavy-Hex Cores and Non-Grid Core Graphs

Question: how much of the foregoing depends on square cores wired in a grid? *Held fixed:* the 64-qubit circuit suite and the physical-qubit count; the core shape and core graph change. *Metric:* geometric-mean EPR and win/loss counts.

We adopt DSABRE's non-grid experiment [11]: the same nine-circuit 64-qubit suite on four IBM 27-qubit heavy-hex tiles. Two things change at once — the *core* becomes irregular (degrees over $\{1, 2, 3\}$, six degree-1 leaves, diameter 12 against a 4×4 grid's 6) and the *core graph* is either a ring, every core of degree 2, or a star with core 0 a hub. Neither compiler is specialised for a heavy-hex tile, and both are given the identical device.

The core graph matters more than the core shape. On the ring (Table VI) the two compilers are within 1% of one another, 86.1 against 85.2: doubling the intra-core diameter and replacing degree-4 interiors with degree-1 leaves disturbs neither. The star does. There PORTER trails by 12%, 113.3 against 101.1, winning two of nine. The core shape is the same in both rows; only the core graph differs.

The initial layout is the dominant observed source of the star gap. The last column of Table VI holds the routing loop against DSABRE's own `SabreLayout` candidates: routed from those, PORTER overtakes DSABRE on *both* core graphs — 82.9 against 85.2 on the ring, 97.1 against 101.1 on the star — and the reversal is larger on the topology it loses under its own layout. The cause is in the layout, and it is specific to the star: PORTER reserves $f_{\min}$ in every core, treating cores as interchangeable, which they are on a grid and a ring and are not where one core is a hub every route must cross. On star Multiplier that reservation opens with free counts $(2, 2, 13, 27)$, the hub pinned at the floor while a whole core sits empty.

This bounds the layout reading rather than extending it: up to 108 physical qubits, across two core shapes and three core graphs, the initial layout separates the two compilers and controlling it erases the difference — a bound that the scale-up series of Section V-D, where the same control does not help, keeps from being read as a general one.

TABLE VII
PROTOCOL TRADE-OFF OVER ALL 24 INSTANCES. "ROUTES" COUNTS ROUTING PASSES; TIMES ARE COMPLETE COMPILE COSTS (GMEAN PER INSTANCE). A THIRD SCHEDULE, WHICH ALSO SEARCHES THE LAYOUT DISCOUNT γ, IS REPORTED IN APPENDIX J.

Protocol	routes	EPR	vs. DSABRE	time (s)	vs. DSABRE
DSABRE	9	171.9	1.00	1.40	1.00
PORTER	4	169.4	0.99	1.68	1.20
PORTER+R	8	156.7	0.91	3.00	2.15

F. The EPR/Time Operating Frontier

Table VII places PORTER's operating points (Section IV-D) against DSABRE, pooled over all twenty-four instances. Each is the previous one plus a search: PORTER selects the horizon and PORTER+R refines backward from the route it selects. Because each schedule contains the one before it and keeps the best result by EPR, the column is monotone by construction, and neither can report more EPR than routing once at the configured horizon.

The trade is time against EPR pairs. PORTER costs 0.99× DSABRE's EPR at 1.20× its compile time; refinement extends the EPR lead to 0.91× for 2.15× the time, on eight routes against DSABRE's nine.

All times are *complete* compile costs — layout generation plus every route over every candidate, not the winning route alone. PORTER routes fewer times than DSABRE at every operating point, four against nine and eight against nine, and is still the slower of the two: its routes are the dearer ones, because each enumerates whole plans, forward- simulates packets and searches two tiers of capacity relief. Compile time is therefore not an argument for PORTER on these suites — we report it because the EPR figures are only meaningful beside the budget that produced them. It does become one at

scale, where the same four routes run in $0.50\times$ DSABRE's complete cost (Section V-D).

VI. Discussion and Future Work

The two compilers are complementary, and a portfolio beats either. PORTER wins ten of the twenty-four instances against DSABRE and loses twelve, PORTER+R eleven and eleven (Table II), so neither dominates instance by instance even though their pooled means are level (169.4 against 171.9). Taking the lower EPR count of the two per instance gives 155.7, a 9.4% reduction on the better of them, and 150.5 with PORTER+R in place of PORTER, 12.5% below DSABRE. The gain is largest where the two disagree most — 16.4% at 80 qubits against 2.5% at 64 — and it is not an artefact of the two cheapest families: over the eighteen instances excluding GHZ and Graphstate it is still 8.0% and 8.9%.

That gap is what the pooled figure conceals, and it is the clearest evidence that the two designs are not interchangeable: a compiler that merely re-derived DSABRE's decisions by another route would leave a portfolio with nothing to choose between. A 9–12% gain instead means each compiler reaches circuits the other does not, and by wide margins where it does — 80-qubit Multiplier at 2,874 pairs against 4,841, 80-qubit QPEexact at 276 against 347, with DSABRE ahead elsewhere. Which of the differences between them produces any particular win is not something this measurement can say: the two differ at once in action vocabulary, candidate granularity, where preparation cost enters, commitment, initial layout, horizon policy and route budget, and the portfolio ranks outputs rather than separating causes. It is a property of the pair rather than of either compiler: it costs both routes ($2.3\times$ DSABRE's compile time, thirteen routes to nine), and no rule we tested predicts the winner in advance.

Why TeleData-only can beat the full action set. Packets beat the single telegates they degrade into, yet a vocabulary with gate teleportation removed altogether — TeleData and nothing else — is 19.8% cheaper than the full action set over the 80-qubit circuits it completes, and cheaper on every one of them but QAOA, which it loses by 0.6%. The cause is a currency mismatch in the scorer: a packet's gain counts remote CX gates resolved immediately, a TeleData plan's a sum of core-distance reductions discounted over a bounded horizon. Nothing establishes the two are commensurate, and where a teleport repays past the horizon the comparator cannot see it. That is evidence the mismatch costs pairs at scale, not that spreading one pair over a run of gates is a bad idea.

A. Future Work

On the compiler itself. Four openings, the first a completion guarantee to match DSABRE's. Adopting its recovery wholesale is the obvious route, and the capacity half is nearly in place — both treat free slots as a legality condition with the same floor of two, and both relieve by breadth-first search to a reservoir (Section VII). What DSABRE adds is atomicity: it relays the chain in one all-or-nothing transaction, where PORTER re-plans each hop through the ordinary plan machinery, which is what lets the opportunistic tier compete with normal work at all (Section III-D). Nor would it buy what is missing. PORTER never reaches the capacity deadlock that proof rules out, and the one failure we observe is a local-SWAP livelock — heavy-hex star Multiplier at a fixed horizon — with capacity healthy throughout. The guarantee needs a progress argument for the intra-core stage, not a stronger capacity transaction. One configuration is worth trying first, because two of our results point at it: DSABRE's proof is written for a teleport-only router, and a teleport-only vocabulary is the cheapest we measured at 80 qubits, so a PORTER restricted to TeleData is both where the guarantee is likeliest to transfer and where the pairs are — though not for free, since that configuration completes twenty-one of the twenty-four instances here.

The other three: a horizon that scales with the core count K — L_{tp} is an absolute gate count, so a plan at twenty cores is scored over proportionally less of the circuit than at four, and every scale-up instance picks the longest candidate offered; a circuit statistic that predicts the horizon well enough to replace the portfolio with a single route — the horizon acts on one moving qubit over a bounded depth, so a per-qubit or horizon-limited measure is the obvious candidate and is untried, and without one the horizon stays a portfolio whose candidate set was fixed on the corpus it is measured on (Section IV-D1); and a comparator that weighs a resolved gate against a latent distance reduction on one scale, which the vocabulary result indicates the present one does not. The mismatch is confined to total itself: the selection key of Section III-E no longer carries a separate discounted term, so a scale on which a resolved gate and a latent distance reduction are commensurable would have to come from the summation, not from a tie-break.

Beyond EPR count. The trace PORTER emits is unscheduled, so a lower pair count does not by itself imply a shorter makespan or a higher-fidelity result. A scheduling pass should batch inter-core actions whose links, ports and preparation SWAPs do not conflict, and account for entanglement-generation rate, storage lifetime and gate error [13], [14]; the objective should stay multi-objective, since pair count, depth and fidelity trade off in hardware-dependent ways. Lifetime bears on PORTER in particular: a packet holds one cat resource open across a whole run, so a lifetime bound caps packet length in a way our cost model does not, and the vocabulary result above would have to be re-read under it.

Richer interconnects. Every device evaluated here gives adjacent cores a single link and each core one to four ports. Widening that is the free variable: a second link turns an occupied port from an obstacle into an alternative, relieving preparation cost first, and it interacts with the capacity discipline, since more ports mean more ways to spend and to recover a free slot. Where the return on added links falls off against their hardware cost is a question for interconnect co-design.

VII. Related Work

TeleSABRE [10] is the published state of the art for this cost model, supporting teledata and gate teleportation on

SABRE's extended-set scoring. The two differ chiefly in decision structure. TELESABRE admits an inter-core candidate only when it is already executable — operand adjacent to a free communication port, a spare slot at the destination — and scores it by the layout energy it leaves behind, a contracted-communication-graph Dijkstra distance over the front and extended sets. Staging is therefore carried by ordinary SWAPs, pooled with those candidates and scored the same way, so each iteration's minimum may be a SWAP that shortens a route or a remote action that resolves a gate. PORTER keeps local work strictly ahead of inter-core work, scores whole plans independently of preparation cost, generates preparation SWAPs only for the plan it commits to, adds the packet primitive, and treats capacity as a legality condition with its own relief search rather than as TELESABRE's safety-valve cap.

DSABRE [11] attacks the same problem and is a close relative. Both DSABRE and PORTER rank moves by SABRE's front-plus-lookahead Δ score, both are strictly locality-first, scoring an inter-core candidate only once no front gate is intra-core, both promote capacity from a scoring penalty to a legality condition with the same floor of two free slots, and both manufacture capacity by relaying a free slot from a reservoir along a BFS path rather than evicting arbitrarily. Four differences remain: PORTER's action vocabulary is larger; its unit of commitment is a whole plan rather than one hop; preparation cost enters as a tiebreaker rather than additively; and its initial layout is one deterministic construction rather than a best-of-N search. A fifth is a gap rather than a design choice, and it runs DSABRE's way: its safe mode carries a termination-with-success proof under stated architectural hypotheses, so it guarantees *completion*, where PORTER guarantees only correctness and termination by construction (Section III-H) and has completion empirically, no instance being left unrouted under the selected-horizon protocol of Section IV-D1.

Other approaches. Partition-first compilers assign qubits to cores by hypergraph or community partitioning [15]–[17] and route the residue; `pytket-dqc` [16] is the mature representative, compared against in Appendix B under the caveat that its e-bit unit assumes long-lived entanglement and different communication memory. AutoComm [8] identifies gate sequences that can share one entangled resource, which PORTER's packet applies inside the routing loop rather than as a pre-pass (Section II-C). SABRE [12] supplies the scoring skeleton all three inherit.

Table VIII places these systems on the axes that matter here.

VIII. CONCLUSION

PORTER is a deterministic DQC compiler built on a plan-oriented remote stage: it scores a complete inter-core action on the gate progress it buys, before any preparation SWAP for it exists, and commits before preparing. Around that it places a capacity discipline with two relief tiers, a deterministic initial layout, and a teleport-lookahead horizon chosen per circuit from a fixed four-value portfolio.

Three conclusions are supported. First, on the evaluated MQT-Bench suites PORTER uses substantially fewer ($>40\%$) EPR pairs than published TELESABRE and completes all twenty-four instances, including the seven on which TELESABRE does not converge. Second, against DSABRE it is competitive on those same suites, $0.99\times$ its pooled EPR at $1.20\times$ its compile time under each compiler's own protocol, and its advantage grows with the machine, reaching $0.69\times$ on the 100–360-qubit scale-up series. Third, controlled experiments locate where these results come from rather than leaving them to the system as a whole, each by its own kind of evidence: an ablation shows capacity relief is what lets a deterministic-greedy design complete in practice; holding the initial layout fixed shows it, not the routing loop, accounts for most of the spread between the two compilers on those suites; and routing the scale-up series at a fixed horizon instead of a selected one turns $0.69\times$ into $1.45\times$, which is what carries PORTER past DSABRE as the machine grows. The two compilers also prove complementary rather than ranked: neither wins more than eleven of the twenty-four instances, and taking the better of the two per instance reaches 12.5% below DSABRE alone (Section VI).

ACKNOWLEDGMENTS

This work was partially supported by the Natural Science Foundation of China (Grant No. 12471437). The authors used Anthropic's Claude (via Claude Code) to assist with the PORTER implementation, the replay verifier and comparison harness, and the drafting of this manuscript. All AI-generated content was reviewed and verified by the authors, who take full responsibility for the correctness and originality of the work.

REFERENCES

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.

[3] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

TABLE VIII
DESIGN AXES OF THE COMPILERS DISCUSSED HERE. "COMMITMENT" IS THE UNIT A SCORING DECISION IS TAKEN OVER. `PYTKET-DQC` IS PARTITION-FIRST AND DOES NOT ROUTE WITH THESE PRIMITIVES, SO SEVERAL AXES DO NOT APPLY.

	TELESABRE	DSABRE	pytket-dqc	PORTER
Paradigm	online	online	partition-first	online
Primitives	data, gate	data	(e-bit)	data, gate, packet
Local/remote	one pool	staged	—	staged
Commitment	one move	one hop	whole plan	whole plan
Prep. cost	in score	in score	—	tie-break
Layout	Hungarian, 3 seeds	SabreLayout, 3 seeds	hypergraph partition	one deterministic
Capacity	cap	legality + relay	partition	legality + relief
Termination	—	with-success proof	—	correct, terminates
Metric	EPR	EPR	e-bits	EPR

[4] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.
[5] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.
[6] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.
[7] A. Yimsiriwattana and S. J. Lomonaco Jr., "Generalized GHZ states and distributed quantum computing," *arXiv preprint quant-ph/0402148*, 2004.
[8] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.
[9] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.
[10] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.
[11] S. Li, "dSABRE: A SABRE-style router for multi-core distributed quantum computers," 2026, manuscript, Centre for Quantum Software and Information, University of Technology Sydney. Code: https://github.com/ebony72/dsabre.
[12] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.
[13] J. M. Baker, C. Duckering, A. Hoover, and F. T. Chong, "Time-sliced quantum circuit partitioning for modular architectures," in *Proceedings of the 17th ACM International Conference on Computing Frontiers (CF)*, 2020, pp. 98–107.
[14] D. Ferrari, A. S. Cacciapuoti, M. Amoretti, and M. Caleffi, "Compiler design for distributed quantum computing," *IEEE Transactions on Quantum Engineering*, vol. 2, pp. 1–20, 2021.
[15] P. Escofet, M. Bandic, C. G. Almudever, and S. Feld, "HQA: A hungarian-based qubit assignment for multi-core quantum computers," in *Proceedings of the IEEE Quantum Software Conference*, 2023.
[16] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.
[17] O. Daei, K. Navi, and M. Zomorodi-Moghadam, "Optimized quantum circuit partitioning," in *International Journal of Theoretical Physics*, vol. 59, 2020, pp. 3804–3820.



Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.